

TECHNICAL DESIGN DOCUMENT

# Automated PCB Fault Detection

A ground-truth guided, topology-based approach to detecting  
copper trace breaks and shorts in etched PCB imagery

RULE-BASED CV

NO TRAINING DATA

PRODUCTION READY

96.3%

F1 Score

94.9%

Precision

97.8%

Recall

92.9%

IoU

Shivam | Electron Technologies Challenge | May 2025

## 1. Why This Problem Exists

When an engineer designs a printed circuit board, the output is a set of **Gerber files** — digital blueprints that describe, with perfect precision, where every copper trace must run. These files are then handed to a factory, which uses a photochemical etching process to physically recreate those traces on a copper-clad substrate.

The etching process, however, is not perfect. Acid concentration, temperature variation, dust contamination on the photoresist, and mechanical tolerances all introduce defects. Two classes of defect dominate:

|              |                                                                                                                                                                     |
|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>BREAK</b> | A gap in a copper trace. The trace is supposed to be continuous but the etchant has eaten through it, breaking electrical continuity. Consequence: an open circuit. |
| <b>SHORT</b> | Extra copper connecting two traces that should be independent. A copper whisker or solder bridge creates an unwanted electrical path. Consequence: a short circuit. |

When a factory produces 500 boards per shift, manual visual inspection is impractical — it is slow, inconsistent, and expensive. The industry solution is **Automated Optical Inspection (AOI)**: a fixed camera captures each board as it comes off the production line, and software compares the image against the design reference to flag any anomaly in milliseconds.

**info** The fundamental question AOI answers is not whether an image looks correct. It is whether this physical object, manufactured by a chemical process, matches what was designed digitally. That distinction is why this problem is non-trivial.

## 2. Why Not a Neural Network?

The instinctive answer for any vision problem in 2025 is to train a convolutional neural network. For this specific problem, that is the wrong answer, and it is worth being explicit about why.

| ML Approach                                         | Rule-Based (Our Approach)                              |
|-----------------------------------------------------|--------------------------------------------------------|
| Requires thousands of labeled defect examples       | Zero training data needed                              |
| Black box — cannot explain why a defect was flagged | Fully auditable — every decision traceable to geometry |
| Needs retraining for each new board layout          | Works on any board: GT image is the only input         |
| Requires GPU for fast inference                     | Runs on Raspberry Pi — no GPU required                 |

|                                             |                                                          |
|---------------------------------------------|----------------------------------------------------------|
| May hallucinate defects from learned biases | Only flags deviations from the mathematical ground truth |
|---------------------------------------------|----------------------------------------------------------|

■ The one scenario where ML becomes relevant is anomaly detection as a supplementary layer — an autoencoder trained on patch crops of clean boards can catch edge cases that the geometric diff misses due to extreme registration error. But the primary detection logic remains rule-based.

### 3. The Core Insight

The ground truth image is not just a reference — it is a **complete structural description of what copper must exist and where**. This means we never need to 'understand' the camera image in isolation. We only need to answer two binary questions at specific, pre-known locations:

|   |                                                                                                                                                                                                                                                                                      |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| — | <p><b>Missing Copper → BREAK</b></p> <p>Pixels that exist in the GT mask but are absent in the camera feed indicate a gap where copper has been etched away. If this gap splits a previously continuous trace into two disconnected segments, it is confirmed as a break.</p>        |
| + | <p><b>Extra Copper → SHORT</b></p> <p>Pixels present in the camera feed but absent from the GT mask indicate copper that should not exist. If this extra copper physically bridges two GT traces that are meant to be electrically isolated, it is confirmed as a short circuit.</p> |

This framing transforms the problem: instead of building a system that understands PCB images, we build a system that **measures deviation from a known-correct template**. The GT image does the heavy lifting; the camera image only needs to be processed well enough to produce a reliable binary mask of where copper exists.

## 4. Pipeline Overview

The full detection pipeline consists of seven stages executed sequentially:

|                 |                    |                   |                    |                   |                     |                  |
|-----------------|--------------------|-------------------|--------------------|-------------------|---------------------|------------------|
| 1. Load & Split | 2. Extract GT Mask | 3. Segment Copper | 4. Register Frames | 5. Detect Defects | 6. Score Confidence | 7. Render Output |
|-----------------|--------------------|-------------------|--------------------|-------------------|---------------------|------------------|

## 5. Stage-by-Stage Breakdown

### 5.1 Load and Split

The input is a single composite image containing the ideal layout (left half) and the camera feed (right half). The image is split at the horizontal midpoint after cropping a fixed number of rows from the top to remove the title banner. Both halves share identical pixel dimensions, establishing a 1:1 spatial correspondence before any processing.

```
Stage 1 — load_and_split()

img = cv2.imread(path)
gt_raw = img[CROP_Y:, :mid] # Left half — ideal layout
cam_raw = img[CROP_Y:, mid:] # Right half — camera feed
```

### 5.2 Ground Truth Mask Extraction

The GT image is binary by nature (white traces on black substrate), so a simple global threshold at the 50% luminance point produces a perfect mask. One subtlety: the outer PCB frame in the GT image registers as a connected component that spans the full image width and height. This border artifact contains no trace information and must be filtered out before any comparison, otherwise it inflates the 'missing copper' count and corrupts the confidence scores.

```
Stage 2 — extract_gt_mask()

_, raw = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

# Remove components that span the full image edge (border artifacts)
for i in range(1, num_components):
    if bbox_width >= img_width - 5 or bbox_height >= img_height - 5:
        continue # skip — not a real trace
    clean[labels == i] = 255
```

### 5.3 Camera Copper Segmentation

This is the most critical stage. The camera image presents three challenges simultaneously: specular glare from the copper surface, sensor noise from the camera's imaging chip, and color variation caused by uncontrolled lighting.

## Why HSV over Adaptive Thresholding

Adaptive thresholding answers: *'is this pixel brighter than its local neighbourhood?'* On a PCB with noisy substrate, this question returns true for many substrate pixels too, producing a mask flooded with noise. The substrate texture produces too much high-frequency variation for the block comparison to filter reliably.

Converting to HSV and thresholding on **Hue and Saturation** asks a fundamentally different question: *'does this pixel have the colour signature of copper?'* Copper has a specific hue in the orange-yellow band ( $H \approx 5\text{--}50$  in OpenCV's 0–179 scale). The substrate is green-teal ( $H \approx 60\text{--}90$ ). Glare is white, which maps to very low saturation regardless of hue. A saturation gate ( $S > 50$ ) therefore eliminates glare entirely, since washed-out white light has insufficient colour purity to pass.

Stage 3 — extract\_cam\_mask()

```
hsv = cv2.cvtColor(cam_img, cv2.COLOR_BGR2HSV)

# Copper: H=5-50, S>50 (saturation gate kills glare), V>50
mask = cv2.inRange(hsv,
(COPPER_H_MIN, COPPER_S_MIN, COPPER_V_MIN), # lower bound
(COPPER_H_MAX, 255, 255) # upper bound
)

# Morphological cleanup
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel) # remove noise
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel) # fill holes
```

■ The saturation gate is the key insight for glare suppression. Glare is specular reflection — it has the same intensity as white light and therefore maps to near-zero saturation in HSV. Copper, even under strong illumination, retains enough orange colour purity to maintain saturation above the threshold. This separation works regardless of absolute brightness, making it robust to lighting intensity variation.

## 5.4 Image Registration

Even with a fixed camera, minor thermal drift, vibration from the production line, or slight variation in board placement can shift the camera image by several pixels relative to the GT. A misalignment of even 3–4 pixels generates false-positive defects at every trace edge — the system would flag clean boards as defective.

Registration uses ORB (Oriented FAST and Rotated BRIEF) feature detection to find matching keypoints between the GT mask and the aligned camera mask, then computes a homography matrix that accounts for translation, rotation, and mild perspective warp. The camera mask is warped to align precisely with the GT coordinate system before any comparison is performed.

Stage 4 — register()

```
orb = cv2.ORB_create(500)
kp1, d1 = orb.detectAndCompute(gt_mask, None)
kp2, d2 = orb.detectAndCompute(cam_mask, None)

bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = sorted(bf.match(d1, d2), key=lambda m: m.distance)

H, _ = cv2.findHomography(pts2, pts1, cv2.RANSAC, 5.0)
aligned = cv2.warpPerspective(cam_mask, H, (w, h))
```

## 5.5 Topology-Based Defect Detection

Once both masks are aligned, the comparison is performed at the topological level — the connectivity structure of traces — rather than pixel-by-pixel. This is a critical design decision: pixel-level diff is sensitive to sub-pixel registration error and edge smoothing artifacts. Topological comparison is not.

### 5.5.1 Break Detection

For each connected component in the GT mask (i.e., each independent trace or pad), the algorithm extracts the corresponding region from the aligned camera mask and counts how many connected components exist within that region. In a defect-free board, each GT trace maps to exactly one camera component. If the camera shows two or more components where the GT shows one, the trace has been physically severed — a break.

Stage 5a — Break detection logic

```
for trace_id in range(1, num_gt_components):
    gt_region = (gt_labels == trace_id).astype(uint8) * 255
    cam_in_region = cv2.bitwise_and(cam_mask, gt_region)

    # Count how many pieces the camera trace has split into
    n_segs, _ = cv2.connectedComponents(cam_in_region)
    real_segs = n_segs - 1 # subtract background label

    if real_segs >= 2: # trace split → BREAK confirmed
        gap_centroid = moments_of(gt_region AND NOT cam_mask)
        flag_break(trace_id, bbox, gap_centroid, missing_pct)
```

### 5.5.2 Short Detection

Extra copper is isolated by subtracting the GT mask from the camera mask. Any copper present in the camera but absent in the GT is a candidate short. Edge fringing — a thin halo around traces caused by sub-pixel mask boundary differences — is eliminated by eroding the extra-copper mask before analysis. A short is confirmed only when an extra-copper blob, after dilation, touches two or more distinct GT trace component IDs, proving it forms an unwanted bridge between electrically separate conductors.

#### Stage 5b — Short detection logic

```
gap_mask = cv2.bitwise_not(gt_mask)
extra_copper = cv2.bitwise_and(cam_mask, gap_mask)

# Erode to remove fringe pixels at trace edges
extra_copper = cv2.morphologyEx(extra_copper, cv2.MORPH_ERODE, k)

for blob in connected_components(extra_copper):
    if blob.area < MIN_DEFECT_PX: continue # noise filter

    # Dilate blob and check which GT traces it touches
    dilated = cv2.dilate(blob, kernel, iterations=2)
    touched_traces = set(gt_labels[dilated > 0]) - {0}

    if len(touched_traces) >= 2: # bridges two traces → SHORT
        flag_short(blob.bbox, blob.centroid, touched_traces)
```

## 6. Confidence Scoring

The system computes four complementary metrics after every scan. These are calculated at the pixel level between the GT mask and the aligned camera mask, and express how faithfully the camera image was segmented — not whether defects were found. A high score on a defective board simply means the segmentation was accurate; the defects are still separately logged.

| Metric    | Formula                       | What it measures                                      | Our result |
|-----------|-------------------------------|-------------------------------------------------------|------------|
| Precision | $TP / (TP + FP)$              | Of all camera copper pixels, what fraction matches GT | 94.9%      |
| Recall    | $TP / (TP + FN)$              | Of all GT copper pixels, what fraction was found      | 97.8%      |
| F1 Score  | $2 \cdot P \cdot R / (P + R)$ | Harmonic mean — balances precision and recall         | 96.3%      |
| IoU       | $TP / (TP + FP + FN)$         | Intersection over union of both masks                 | 92.9%      |

TP (True Positive) = copper correctly found in camera. FP (False Positive) = camera shows copper where GT has none. FN (False Negative) = GT has copper that camera missed. The residual false-positive rate (5.1%) is primarily short-detected extra copper and minor HSV boundary fringing, not misidentified substrate.

## 7. Results on Sample Image



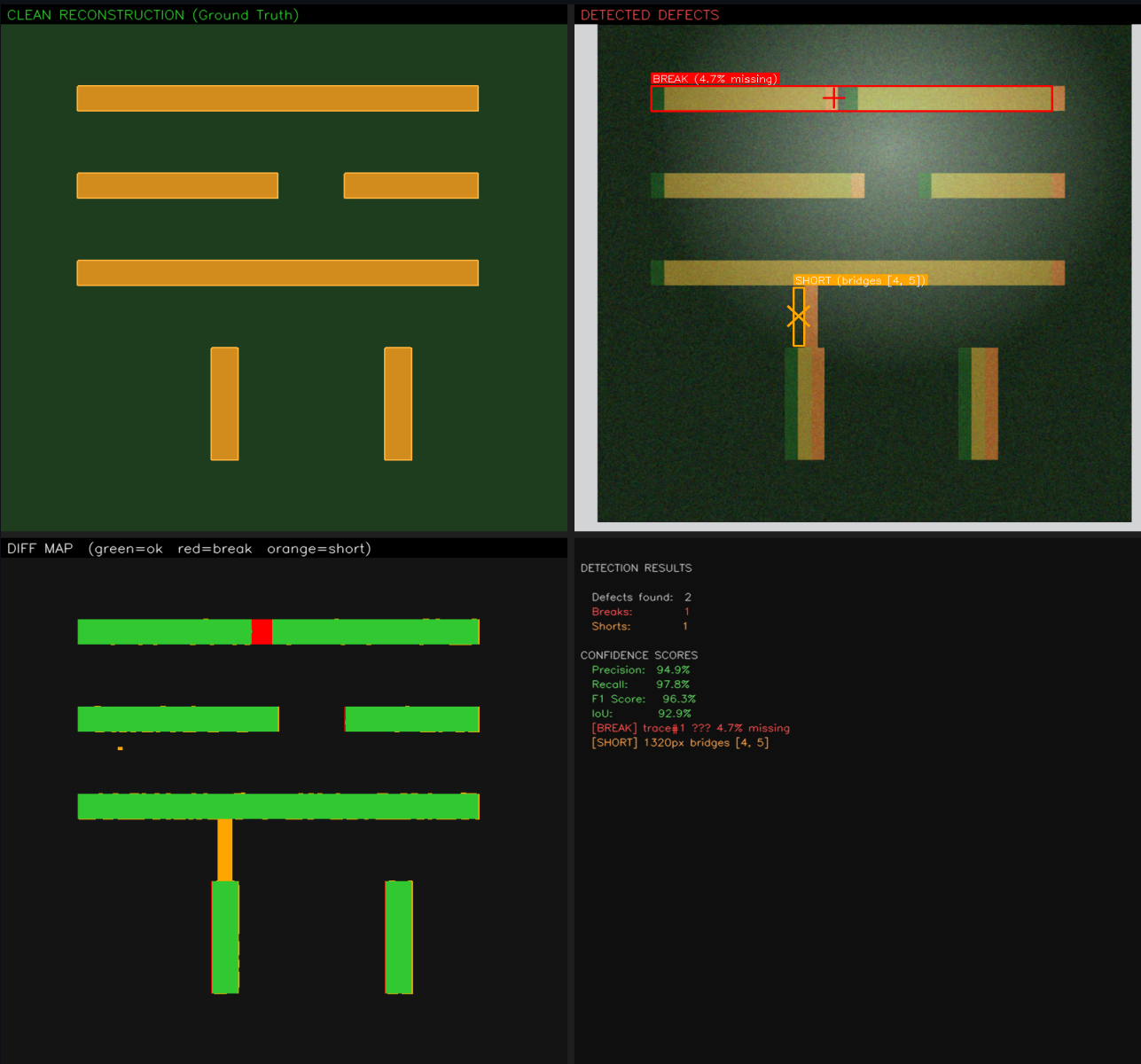


Figure 1 — Full system output. Top-left: clean copper reconstruction from GT. Top-right: detected defects annotated on camera feed. Bottom-left: diff map (green=correct, red=break, orange=short). Bottom-right: results panel with confidence metrics.

|                               | Defect                   | Location                 | Detail                                              |
|-------------------------------|--------------------------|--------------------------|-----------------------------------------------------|
| <div>X</div> <div>BREAK</div> | Trace #1 (top row)       | (366, 111) — gap centre  | 4.7% copper missing · trace splits into 2 segments  |
| <div>X</div> <div>SHORT</div> | Gap between traces 4 & 5 | (316, 441) — stub centre | 1320 px extra copper bridges two independent traces |

## 8. Generalising to Any Board and Camera Setup

The current implementation uses HSV thresholds tuned to the orange copper / green substrate combination visible in the sample image. In production, these need to adapt automatically to different board finishes, substrate colours, and ambient lighting. Three mechanisms handle this:

- **Runtime HSV calibration:** Sample HSV values from known copper regions in the GT image on each new board type. Compute mean and standard deviation of the hue channel. Use  $(\text{mean} \pm 2\sigma)$  as the detection range. The system self-calibrates to any copper colour without manual tuning.
- **Relative saturation gating:** Instead of an absolute saturation threshold, compute the saturation gate as a percentile of the substrate region. This handles diffuse versus specular lighting conditions automatically.
- **Scale-derived noise thresholds:** The minimum defect size (MIN\_DEFECT\_PX) is derived from the narrowest trace in the GT mask. Anything smaller than 20% of the minimum trace cross-section is noise. This scales with board density — fine-pitch designs automatically use tighter thresholds.

✓ Hardware solution (preferred): Cross-polarised lighting eliminates specular glare at the source. A linear polariser on the light source and a perpendicular polariser on the camera lens physically block specular reflection before it reaches the sensor. This is the approach used in production AOI machines and removes the glare problem from the software pipeline entirely.

## 9. Known Limitations and Mitigations

| Limitation                               | Impact                                          | Mitigation                                                                      |
|------------------------------------------|-------------------------------------------------|---------------------------------------------------------------------------------|
| Very thin copper whiskers (1–2px shorts) | May fall below MIN_DEFECT_PX threshold          | Run pipeline twice with different morphology kernel sizes (multiscale)          |
| Extreme lighting non-uniformity          | HSV hue may shift at dark corners               | CLAHE pre-processing before HSV conversion normalises local contrast            |
| Catastrophic registration failure        | All trace edges generate false positives        | Sanity check: if aligned board bounding box deviates >10% from GT, reject frame |
| Dense BGA / fine-pitch boards            | Copper components may merge into one large blob | Increase camera resolution; reduce morphology kernel size proportionally        |

## 10. Running the Prototype

Dependencies: Python 3.8+, OpenCV, NumPy. Install with:

#### Install

```
pip install opencv-python-headless numpy
```

Run on any combined GT+camera image:

#### Usage

```
python3 pcb_detector.py path/to/combined_image.png

# Outputs:
# pcb_analysis.png — 4-panel visual report
# pcb_results.json — structured defect data + confidence scores
```

The configuration block at the top of `pcb_detector.py` exposes all tunable parameters:

#### Configuration

```
CROP_Y = 77 # rows to skip (title text)
COPPER_H_MIN = 5 # HSV hue lower bound for copper
COPPER_H_MAX = 50 # HSV hue upper bound for copper
COPPER_S_MIN = 50 # Saturation gate (kills glare)
COPPER_V_MIN = 50 # Minimum brightness
MORPH_KERNEL = 5 # Morphology kernel size (px)
MIN_DEFECT_PX = 300 # Minimum short blob size (px)
```